Software Requirements Specification

for

Smart Waste Collection Management System

KIET Deemed To Be University

Computer Science Engineering

Prepared by :

Tanishka Singh - Leader (Product Manager)
Subhonita Sarkar - Team Member 1 (Developer)
Shubh Seth - Team Member 2 (Developer)
Shubham Singh Rajput - Team Member 3 (Tester)

Table of Contents

Table of Contents **2**
Revision History **3**
1. Introduction **4**
1.1 Purpose 4
1.2 Document Conventions 4
1.3 Intended Audience and Reading Suggestions 4
1.4 Product Scope 4
1.5 References 4
2. Overall Description **5**
2.1 Product Perspective 5
2.2 Product Functions 5
2.3 User Classes and Characteristics 5
2.4 Operating Environment 5
2.5 Design and Implementation Constraints 5
2.6 User Documentation 6
2.7 Assumptions and Dependencies 6
3. External Interface Requirements **6**
3.1 User Interfaces 6
3.2 Hardware Interfaces 6
3.3 Software Interfaces 6
3.4 Communications Interfaces 7
4. System Features **7**
4.1 Bin Fill-Level Monitoring 7
4.2 Threshold Alert and Notification 7
4.3 Route Optimization 8
4.4 Admin Dashboard and Analytics 8
4.5 Citizen Feedback and Complaint Reporting 8
5. Other Nonfunctional Requirements **9**
5.1 Performance Requirements 9
5.2 Safety Requirements 9
5.3 Security Requirements 9
5.4 Software Quality Attributes 9
5.5 Business Rules 9
6. Other Requirements **10**
Appendix A: Glossary **10**
Appendix B: Analysis Models **10**
Appendix C: To Be Determined List **10**

Revision History

Name	Date	Reason For Changes	Version
Tanishka Singh	05-Aug-2026	Initial draft after Lab 1 — Problem identification & scope	0.1
Subhonita Sarkar	19-Aug-2026	Added System Features and Functional Requirements after Lab 2	0.2
Shubh Seth	02-Sep-2026	Added Nonfunctional Requirements and Interfaces after Lab 3	0.3
Shubham Singh Rajput	14-Sep-2026	Final review, testing feedback incorporated, and formatting finalized	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements of the Smart Waste Collection Management System, Version 1.0. The purpose of this document is to define, in a clear and unambiguous manner, what the system is expected to do so that it can guide the design, development, and testing of the project. This SRS covers the complete system, including the sensor-based bin monitoring hardware, the backend server, the admin web dashboard, and the citizen/driver mobile application.

1.2 Document Conventions

This document follows the standard IEEE SRS template. Functional requirements are uniquely identified using the format REQ-<Feature>-<Number> (e.g., REQ-BM-1). Requirements are assumed to carry equal priority unless a specific priority (High/Medium/Low) is explicitly mentioned against a system feature. Headings are numbered hierarchically (e.g., 4.1, 4.1.1) to allow direct reference during reviews.

1.3 Intended Audience and Reading Suggestions

This document is intended for the project guide/evaluator, the development team members, and testers involved in the project. Section 1 and 2 provide a high-level overview suitable for all readers. Section 3 and 4 are most relevant to developers and designers implementing the interfaces and features. Section 5 is relevant to testers and reviewers verifying quality attributes. Readers are advised to read Sections 1 and 2 first for context, followed by Sections 3-6 for detailed requirements.

1.4 Product Scope

The Smart Waste Collection Management System is an IoT-based system designed to automate and optimize municipal solid-waste collection. The system uses ultrasonic sensors mounted on waste bins to measure fill-levels in real time, transmits this data to a cloud backend, and presents it on an administrative dashboard used to generate optimized collection routes. The system also allows citizens to report overflowing bins. The goal is to reduce unnecessary collection trips, prevent bin overflow, lower fuel and manpower cost, and improve overall urban sanitation, in line with Smart City objectives.

1.5 References

- IEEE Software Requirements Specification Template (Karl E. Wiegers), as provided by the course instructor.
- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- Smart Waste Collection Management System — Problem Identification & Solution Document (Lab 1 submission).

2. Overall Description

2.1 Product Perspective

The Smart Waste Collection Management System is a new, self-contained system and is not a replacement for any existing automated system currently in use, since the target municipal process is presently manual. The system consists of four major components: (1) IoT sensor nodes installed on waste bins, (2) a cloud backend server, (3) a web-based administrator dashboard, and (4) a mobile application used by collection staff and citizens. These components communicate with each other over Wi-Fi/GPRS and REST APIs, as illustrated conceptually below: Sensor Node → Cloud Backend Server → [Admin Dashboard | Mobile App]

2.2 Product Functions

- Continuously monitor and report the fill-level of each waste bin.
- Send threshold-based alerts when a bin approaches full capacity.
- Generate an optimized collection route covering bins that require pickup.
- Display real-time bin status and historical analytics on an admin dashboard.
- Allow citizens to report overflowing bins or request unscheduled pickup.
- Maintain historical records for auditing and long-term planning.

2.3 User Classes and Characteristics

User Class	Characteristics
Municipal Administrator	Manages the system, views dashboard/analytics, configures thresholds; assumed to have basic computer literacy.
Collection Staff / Driver	Uses the mobile app to view assigned routes; assumed to have minimal technical expertise, needs a simple interface.
Citizen	Uses the mobile app or web form to report issues; general public, no technical expertise assumed.
System Administrator	Maintains servers, sensors, and connectivity; technically proficient.

2.4 Operating Environment

- Sensor nodes: Microcontroller (ESP32/Arduino) with Wi-Fi/GPRS module, operating outdoors, powered by battery/solar.
- Backend server: Cloud-hosted (e.g., AWS/Firebase/Render), Linux-based environment.
- Admin Dashboard: Any modern web browser (Chrome, Firefox, Edge) on desktop/laptop.
- Mobile App: Android 8.0+ (and/or iOS 13+), built using Flutter/React Native.

2.5 Design and Implementation Constraints

- Sensor nodes must operate on low power due to limited/solar-based power supply in outdoor bins.
- Data transmission must work over intermittent/low-bandwidth GPRS networks in certain areas.
- The system must use open/low-cost APIs (e.g., OpenStreetMap) to keep the project cost-effective for an academic prototype.
- Route optimization must complete within a few seconds to remain usable for daily planning.

2.6 User Documentation

The following user documentation will be delivered along with the software: an Admin Dashboard user guide, a mobile app quick-start guide for drivers and citizens, and an in-app help/FAQ section. Documentation will be delivered in PDF and in-app formats.

2.7 Assumptions and Dependencies

- It is assumed that bins selected for automation have stable connectivity (Wi-Fi/GPRS coverage) at their location.
- It is assumed that municipal authorities will provide bin location and route data required for initial system setup.
- The system depends on third-party mapping/routing APIs (e.g., Google Maps or OpenStreetMap) for route generation.
- The project assumes availability of ultrasonic sensors and microcontroller hardware within the academic lab for prototyping.

3. External Interface Requirements

3.1 User Interfaces

- Admin Dashboard: map view showing all bins color-coded by fill-level (green/yellow/red), a side panel listing bins needing collection, and an analytics/reports tab.
- Mobile App (Driver): list/map view of the day's optimized route with the ability to mark a bin as "collected".
- Mobile App (Citizen): a simple form to report an overflowing bin with location auto-detected and an optional photo upload.
- All interfaces will follow a consistent color scheme and include a help icon on every screen.

3.2 Hardware Interfaces

- Ultrasonic sensor (HC-SR04 or equivalent) interfaced with the microcontroller via GPIO pins to measure bin fill-level.
- Microcontroller (ESP32/Arduino) interfaced with a Wi-Fi/GSM module (e.g., SIM800L) for data transmission to the cloud server.
- Optional GPS module on collection vehicles for live vehicle-location tracking (Phase 2).

3.3 Software Interfaces

Software Component	Purpose / Interface
Backend Server (Node.js/Django)	Exposes REST APIs for sensor data ingestion, route computation, and dashboard queries.
Database (MongoDB/PostgreSQL)	Stores bin data, sensor logs, user data, and complaint records.
Mapping API (Google Maps/OSM)	Used for route optimization and map visualization on dashboard and mobile app.
Push Notification Service (Firebase Cloud Messaging)	Sends threshold alerts to admin/driver apps.

3.4 Communications Interfaces

- Sensor-to-server communication uses HTTP/MQTT over Wi-Fi or GPRS.
- Dashboard and mobile app communicate with the backend using HTTPS REST APIs in JSON format.
- Data is transmitted at fixed intervals (e.g., every 15 minutes) to balance real-time accuracy and power/bandwidth consumption.

4. System Features

4.1 Bin Fill-Level Monitoring

4.1.1 Description and Priority

Continuously measures and reports the fill-level of each waste bin using an ultrasonic sensor. Priority: High.

4.1.2 Stimulus/Response Sequences

The sensor periodically measures the distance from the sensor to the waste surface → the microcontroller converts this to a fill percentage → the value is transmitted to the server → the server updates the bin's status on the dashboard in real time.

4.1.3 Functional Requirements

- **REQ-BM-1:** The system shall measure the fill-level of each bin at a configurable interval (default: every 15 minutes).
- **REQ-BM-2:** The system shall transmit fill-level data to the backend server along with a bin ID and timestamp.
- **REQ-BM-3:** The system shall handle sensor read errors gracefully by retrying and logging failed readings.

4.2 Threshold Alert and Notification

4.2.1 Description and Priority

Notifies municipal staff when a bin's fill-level crosses a configurable threshold. Priority: High.

4.2.2 Stimulus/Response Sequences

Bin fill-level crosses the configured threshold (e.g., 80%) → the server detects the condition on the next data update → a push notification/alert is sent to the admin dashboard and relevant driver's mobile app.

4.2.3 Functional Requirements

- **REQ-AL-1:** The system shall allow an administrator to configure the alert threshold percentage per bin or globally.
- **REQ-AL-2:** The system shall generate an alert within 5 minutes of a bin crossing the configured threshold.
- **REQ-AL-3:** The system shall log all generated alerts with bin ID and timestamp for audit purposes.

4.3 Route Optimization

4.3.1 Description and Priority

Generates an optimized collection route covering bins that currently require pickup. Priority: High.

4.3.2 Stimulus/Response Sequences

Administrator requests a route (or a scheduled daily trigger fires) → system fetches all bins above the collection threshold → route optimization algorithm computes the shortest route → route is displayed on the dashboard and pushed to the driver's mobile app.

4.3.3 Functional Requirements

- **REQ-RO-1:** The system shall generate an optimized route including all bins currently above the collection threshold.
- **REQ-RO-2:** The system shall complete route computation within 10 seconds for up to 200 bins.
- **REQ-RO-3:** The system shall allow the administrator to manually add or remove a bin from a generated route.

4.4 Admin Dashboard and Analytics

4.4.1 Description and Priority

Provides a web-based dashboard for administrators to monitor bin status and view historical analytics. Priority: Medium.

4.4.2 Stimulus/Response Sequences

Administrator logs into the dashboard → system displays a live map of all bins with color-coded status → administrator can drill down into a specific bin's historical fill-level trend or view a collection-efficiency report.

4.4.3 Functional Requirements

- **REQ-DB-1:** The system shall display all bins on a map, color-coded according to current fill-level status.
- **REQ-DB-2:** The system shall provide a report of the number of collections and average fill-level per bin over a selected date range.
- **REQ-DB-3:** The system shall restrict dashboard access to authenticated administrator accounts.

4.5 Citizen Feedback and Complaint Reporting

4.5.1 Description and Priority

Allows citizens to report an overflowing bin or request an unscheduled pickup. Priority: Medium.

4.5.2 Stimulus/Response Sequences

Citizen opens the mobile app/web form → selects or auto-detects the bin location → submits a complaint with an optional photo → the complaint is logged and forwarded to the administrator dashboard for action.

4.5.3 Functional Requirements

- **REQ-CF-1:** The system shall allow a citizen to submit a complaint with location and an optional photo.

- **REQ-CF-2:** The system shall assign a unique tracking ID to each submitted complaint.
- **REQ-CF-3:** The system shall allow the citizen to view the current status (Pending/Resolved) of their complaint.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- The system shall support at least 500 concurrent sensor nodes without degradation in data-ingestion performance.
- The admin dashboard shall load bin status data within 3 seconds under normal network conditions.
- Route optimization shall complete within 10 seconds for up to 200 bins, as stated in REQ-RO-2.

5.2 Safety Requirements

- Sensor hardware installed on bins shall be weatherproofed and safely enclosed to prevent electrical hazards.
- The system shall not, under any failure condition, prevent manual/emergency waste collection from proceeding.

5.3 Security Requirements

- All communication between the mobile app, dashboard, and backend server shall be encrypted using HTTPS/TLS.
- Administrator accounts shall require authentication (username/password, with optional OTP) before dashboard access is granted.
- Citizen-submitted personal data (e.g., location, contact info) shall be stored securely and used only for complaint resolution.

5.4 Software Quality Attributes

- Usability: The mobile app and dashboard shall be usable by non-technical municipal staff with minimal training.
- Reliability: Sensor data transmission shall have a success rate of at least 95% under normal network conditions.
- Scalability: The backend architecture shall support scaling to additional bins/zones without redesign.
- Maintainability: The codebase shall be modular (separate modules for monitoring, routing, alerts, and dashboard) to simplify future maintenance.

5.5 Business Rules

- Only authenticated administrators may configure alert thresholds or view city-wide analytics.
- A bin shall be included in the next optimized route only if its fill-level is at or above the configured threshold.
- A citizen complaint shall be automatically marked "Resolved" once the corresponding bin is recorded as collected.

6. Other Requirements

No additional legal, internationalization, or reuse requirements have been identified at this stage. This section will be updated if further requirements emerge in subsequent lab sessions.

Appendix A. Glossary

Term	Definition
IoT	Internet of Things — network of physical devices embedded with sensors to collect and exchange data.
Fill-Level	The percentage indicating how full a waste bin currently is, measured by the ultrasonic sensor.
Threshold	A configurable fill-level percentage that triggers an alert (e.g., 80%).
REST API	Representational State Transfer — an architectural style for the web APIs used between app, dashboard, and server.
Route Optimization	The process of computing the most efficient path to collect all bins requiring pickup.

Appendix B. Analysis Models

A high-level system architecture diagram (Sensor Node → Cloud Backend → Admin Dashboard / Mobile App) and a data-flow diagram will be added here as the design phase progresses in subsequent labs.

Appendix C. To Be Determined List

- TBD-1: Final choice of route-optimization algorithm (Nearest-Neighbor heuristic vs. a Vehicle Routing Problem solver).
- TBD-2: Final mapping/routing API provider (Google Maps vs. OpenStreetMap) based on cost constraints.
- TBD-3: Exact default threshold percentage to be finalized after pilot testing.